

Sensors

for Intelligent Multi-Sensor Fusion & Navigation

PHASE 6 — RESEARCH-GRADE NOTES

Trinetra-AI

An Intelligent Multi-Sensor Fusion Framework for GPS-Denied Navigation

The sensors are the eyes of the system. This document treats each one — **accelerometer, gyroscope, magnetometer, IMU, GPS, barometer**, and the future **quantum magnetometer, atomic clock, LiDAR, camera, wheel encoder** — from the standpoint of *intelligent navigation*: how it works physically, how it lies (noise, bias, drift), how to correct it, and exactly where it feeds the Trinetra-AI pipeline. Intuition first, then physics and mathematics, then derivations, then code, then project mapping.

Physical principle → error model → calibration → code → fusion role.

Contents

1	Introduction	2
1.1	What is a sensor?	2
1.2	Why sensors are the foundation of intelligent navigation	2
1.3	Sensors across the mission domains	2
2	Sensor Fundamentals (Metrology)	3
2.1	Accuracy, precision, and error structure	3
2.2	Sensitivity, resolution, scale factor, range	3
2.3	Bandwidth, response time, sampling rate	3
2.4	Non-idealities: bias, drift, noise, hysteresis, and friends	4
2.5	Uncertainty and error types	4
2.6	Metrology reference table	4
3	Accelerometer	5
3.1	Working principle	5
3.2	Axes, units, gravity, free-fall	5
3.3	Noise, bias, drift, temperature, calibration	6
4	Gyroscope	6
4.1	Angular velocity and the Coriolis principle	6
4.2	Bias, random walk, bias instability, Allan variance	6
4.3	Temperature, calibration, and gyro grades	7
5	Magnetometer	7
5.1	Earth’s field, heading, declination, inclination	8
5.2	Hard-iron and soft-iron distortion	8
5.3	Magnetic mapping, localization, and quantum magnetometers	8
6	IMU	9
6.1	Definition and DOF	9
6.2	Coordinate frames and rotation representations	9
6.3	Dead reckoning and integration error	10
7	GPS	10
7.1	Architecture and positioning principle	10
7.2	Coordinates: ECEF, ENU, geodetic	11
7.3	Error sources — why GPS cannot be trusted alone	11
8	Barometer	12

8.1	Pressure, altitude, and the barometric formula	12
8.2	Drift, weather, calibration, noise	12
9	Sensor Errors	13
10	Sensor Calibration	13
10.1	Why, and the three regimes	13
10.2	The general IMU calibration model	14
10.3	Magnetometer ellipsoid calibration	14
11	Sensor Drift	14
12	Sensor Datasets	15
13	Research Papers	16
13.1	Foundational — inertial sensors & navigation	16
13.2	Learned inertial & magnetic navigation (most relevant)	16
14	Sensors inside Trinetra-AI	16
15	Practical Implementation	18
16	Interview & Research Preparation	18
16.1	50 interview questions	18
16.2	30 viva questions	20
16.3	25 research questions	20
16.4	20 coding questions	21
17	Summary, Glossary & Roadmap	22
17.1	Summary	22
17.2	Glossary	22
17.3	Roadmap: Phase 6 → Phases 7, 8, 9 & the Trinetra-AI architecture	22

1 Introduction

1.1 What is a sensor?

A **sensor** is a device that converts a physical quantity (acceleration, rotation rate, magnetic field, pressure) into a measurable signal — usually an electrical voltage that is then sampled and quantised into digital numbers. Formally it implements a *transfer function* $z = h(x) + \text{errors}$, mapping the true world state x to a reading z . **Sensor engineering** is the discipline of designing, modelling, calibrating, and correcting these devices so that h is well-characterised and the errors are bounded and known.

Intuition: a sensor is a translator between the physical world and the computer, and like any translator it introduces its own accent (bias), mishearings (noise), and slow misremembering (drift). Everything in navigation depends on how well we understand that translator.

1.2 Why sensors are the foundation of intelligent navigation

A navigation system can only ever be as good as the sensors feeding it — the *garbage-in, garbage-out* law is absolute. Sensor fusion (Phase 7) and Kalman filtering (Phase 8) are *optimal only when given a correct error model* of each sensor: the covariances **Q**, **R**, the biases, the noise densities. You cannot write those models without understanding the sensor physics. Hence this phase precedes fusion: **fusion is impossible to do well without first understanding sensors**.

1.3 Sensors across the mission domains

Domain	Sensor emphasis	Trinetra-AI parallel
Robotics	IMU + wheel encoders + LiDAR/camera for SLAM	core fusion & odometry
Autonomous vehicles	GNSS + IMU + wheel + camera; multi-path handling	GPS-denied bridging
Aircraft	INS-grade IMU + air-data (baro) + GNSS	drift discipline over long flights
Drones / UAVs	MEMS IMU + baro + mag + GPS; vibration-heavy	lightweight edge fusion
Submarines	INS + doppler velocity log + gravimetry (no GPS underwater)	long-duration dead reckoning
Defense systems	jam/spoof-resistant nav; magnetic & terrain aiding	passive GPS-denied navigation
ISRO / NASA (space)	star trackers, IMU, sun sensors; no GPS beyond MEO	extreme drift discipline
Quantum navigation	quantum magnetometer + atomic clock + INS	the project's north star

Modern GPS-denied programmes make the thesis of this phase concrete: Boeing's 2024 flight tests navigated an aircraft without GPS for hours using *multiple quantum sensors*; SandboxAQ's AQNav and Q-CTRL's Ironstone Opal pair a **quantum magnetometer** with a classical INS and AI de-noising to read Earth's crustal magnetic “landmarks” passively and unjammably. Trinetra-AI is the software/AI layer of exactly this architecture — which is why understanding each sensor's physics and error model is the foundation everything else stands on.

2 Sensor Fundamentals (Metrology)

Before any specific sensor, we need the vocabulary of *measurement science*. These terms are what every datasheet reports and what every error model is built from. We define them in clusters, give the key mathematics, then consolidate in a reference table.

2.1 Accuracy, precision, and error structure

Accuracy is closeness of the mean reading to the true value (freedom from *systematic* error / bias). **Precision** (repeatability) is closeness of repeated readings to *each other* (freedom from *random* error / noise). A sensor can be precise but inaccurate (tight cluster, wrong place) or accurate but imprecise (right on average, scattered).

Any single reading decomposes as

$$z = x + \underbrace{b}_{\text{systematic (bias)}} + \underbrace{\varepsilon}_{\text{random, } \varepsilon \sim \mathcal{N}(0, \sigma^2)}.$$

Accuracy is governed by b (removable by calibration); *precision* by σ (reducible by filtering/averaging: averaging N samples cuts σ by $\sqrt{N}$). The mean-squared error splits as $\text{MSE} = b^2 + \sigma^2$ — the bias–variance decomposition, identical to the one in machine learning.

2.2 Sensitivity, resolution, scale factor, range

Sensitivity is the slope of output vs. input, $S = \frac{\partial z}{\partial x}$ (e.g. mV per g). **Scale-factor error** is a proportional error $z = (1 + k)x$ where k is the deviation of the true sensitivity from nominal. **Resolution** is the smallest change in x that produces a detectable change in z . **Range** is the $[x_{\min}, x_{\max}]$ the sensor can measure; **dynamic range** is their ratio,

$$\text{DR}_{\text{dB}} = 20 \log_{10} \frac{x_{\max}}{x_{\min}}.$$

2.3 Bandwidth, response time, sampling rate

Modelling a sensor’s dynamics as a first-order system, a step input settles with time constant τ , and its -3 dB **bandwidth** is

$$f_{\text{bw}} = \frac{1}{2\pi\tau}.$$

A wider bandwidth (small τ) tracks fast motion but admits more noise; a narrow bandwidth is smooth but sluggish. The **sampling rate** f_s must satisfy Nyquist $f_s > 2f_{\text{bw}}$ (Phase 5) or aliasing corrupts the reading. **Latency** is the delay between the physical event and the

available sample — critical when fusing sensors of different rates.

2.4 Non-idealities: bias, drift, noise, hysteresis, and friends

Term	Definition / math	Navigation consequence
Bias	non-zero output at zero input, b	integrates into drift
Drift	slow time-variation of bias, $b(t)$	the core GPS-denied enemy
Noise	random component, variance σ^2	sets $\mathbf{R}$; limits precision
Cross-sensitivity	one axis responds to another's input	couples errors between axes
Hysteresis	output depends on input <i>history</i> (loop)	path-dependent error
Saturation	output clips beyond range	lost data on hard impacts
Dead zone	no output for small inputs near zero	missed slow motion
Nonlinearity	deviation of h from a straight line	scale error varies with x
Quantisation	rounding to ADC steps, var = $\Delta^2/12$	floor on resolution
Temperature effect	bias/scale vary with T	predictable, compensable drift

2.5 Uncertainty and error types

Systematic error is repeatable and predictable (bias, scale, misalignment) — removable by *calibration*. **Random error** is unpredictable (noise) — reducible only statistically (filtering, averaging). **Measurement uncertainty** is the quantified doubt about a reading, usually a standard deviation or confidence interval, and *propagates* through computations: for $y = f(x_1, \dots)$,

$$\sigma_y^2 \approx \sum_i \left(\frac{\partial f}{\partial x_i} \right)^2 \sigma_{x_i}^2 \quad (\text{first-order propagation of uncertainty}).$$

2.6 Metrology reference table

Property	Symbol / math	Sensor example	Role in Trinetra-AI
Accuracy	bias b	accel zero- g offset	calibrate out; init $\mathbf{R}$
Precision	noise σ	gyro rate noise	sets $\mathbf{R}$; filter/average
Sensitivity	$S = \partial z / \partial x$	mag mV/ μ T	scale conversion
Scale factor	$(1 + k)$	accel gain error	calibration parameter
Resolution	Δ	ADC LSB	feature/precision floor
Dynamic range	$20 \log_{10}(x_{\max}/x_{\min})$	IMU $\pm 16g$	handle impacts & micro-motion
Bandwidth	$f_{\text{bw}} = 1/2\pi\tau$	gyro 100 Hz	window/filter design
Latency	delay t_d	sensor-to-bus	fusion timestamping
Drift	$b(t)$	gyro bias walk	drift-prediction target
Cross-sens. / mis-align.	off-diagonal $\mathbf{M}$	IMU axis skew	calibration matrix
Temperature	$b(T), S(T)$	MEMS thermal drift	temp-compensation model
Uncertainty	σ_y propagation	fused estimate	confidence scoring

```
import numpy as np
def characterize_static(z): # z: (N, axes) static recording
    bias = z.mean(axis=0) # systematic offset
    noise = z.std(axis=0) # 1-sigma random error
```

```
# averaging N samples reduces noise by sqrt(N):
return {"bias": bias, "noise_1sigma": noise,
        "noise_after_avg": noise/np.sqrt(len(z))}
```

Interview prompts. *Distinguish accuracy from precision with a sensor example. Why does averaging reduce noise by $\sqrt{N}$ but not bias? Derive the bandwidth–response-time relation. How does uncertainty propagate when you integrate acceleration to velocity?*

3 Accelerometer

3.1 Working principle

An **accelerometer** measures *specific force* (proper acceleration) — acceleration relative to free-fall — along its sensitive axes. At rest it reads the reaction to gravity ($\approx 9.81 \text{ m/s}^2$ “upward”); in free-fall it reads zero. It does *not* measure coordinate acceleration directly; gravity must be subtracted using orientation.

A proof mass m on a spring (stiffness k) with damping c inside the moving case obeys, for case acceleration a ,

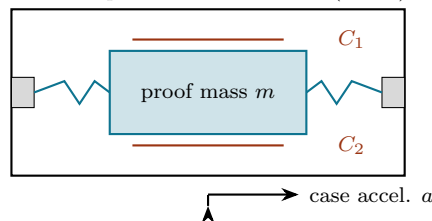
$$m\ddot{x} + c\dot{x} + kx = -ma,$$

where x is mass displacement relative to the case. In steady state ($\ddot{x} = \dot{x} = 0$),

$$x_{\text{static}} = -\frac{m}{k}a = -\frac{a}{\omega_n^2}, \quad \omega_n = \sqrt{k/m}.$$

So displacement is proportional to acceleration — measure x and you measure a . The **sensitivity** $m/k = 1/\omega_n^2$ trades off against **bandwidth** ω_n : a softer spring senses smaller accelerations but responds more slowly. MEMS devices read x *capacitively*.

MEMS capacitive accelerometer (1 axis)



Capacitive readout. The proof mass forms a differential capacitor with fixed plates; displacement x changes C_1, C_2 oppositely, and $\Delta C = C_1 - C_2 \propto x \propto a$. **Piezoelectric** accelerometers instead generate charge under stress (great for high-frequency vibration, poor at DC); **capacitive MEMS** dominate navigation because they measure down to DC (static gravity).

3.2 Axes, units, gravity, free-fall

Three orthogonal axes $[a_x, a_y, a_z]$ in the *body frame*; units m/s^2 or g ($1g = 9.80665 \text{ m/s}^2$). At rest the measured vector has magnitude g and points opposite to gravity — enabling **roll** and **pitch**

estimation

$$\phi = \text{atan2}(a_y, a_z), \quad \theta = \text{atan2}(-a_x, \sqrt{a_y^2 + a_z^2}),$$

but *not yaw* (rotation about gravity leaves the gravity vector unchanged).

3.3 Noise, bias, drift, temperature, calibration

Errors: bias (zero- g offset), velocity random walk (VRW, the integral of white noise), scale-factor and axis-misalignment errors, and temperature-dependent bias. A constant bias b_a double-integrates into a position error $\frac{1}{2}b_a t^2$ — the fastest-growing inertial error. Calibration (six-position tumble test, §10) estimates bias, scale, and misalignment.

In Trinetra-AI the accelerometer (i) provides the gravity reference for roll/pitch, gating gyro drift via a complementary/Kalman filter; (ii) drives dead-reckoning through double integration; and (iii) supplies the vibration spectrum used by the sensor-health module. Its bias is a primary target of the drift-prediction network, since its quadratic error growth dominates over time.

```
import numpy as np
def roll_pitch_from_accel(a): # a: (...) in m/s^2, body frame
    ax, ay, az = a[...,0], a[...,1], a[...,2]
    roll = np.arctan2(ay, az)
    pitch = np.arctan2(-ax, np.hypot(ay, az))
    return roll, pitch # radians
def remove_gravity(a, R_bw, g=9.80665): # R_bw: body->world rotation
    return (R_bw @ a) - np.array([0,0,g]) # linear acceleration in world
```

4 Gyroscope

4.1 Angular velocity and the Coriolis principle

A **gyroscope** measures *angular velocity* $\omega = [\omega_x, \omega_y, \omega_z]$ (rad/s or °/s) about its axes. MEMS gyros sense rotation through the **Coriolis effect** on a vibrating proof mass.

A proof mass driven to oscillate with velocity $\mathbf{v}$ (the *drive* mode) in a frame rotating at $\boldsymbol{\omega}$ experiences a Coriolis acceleration

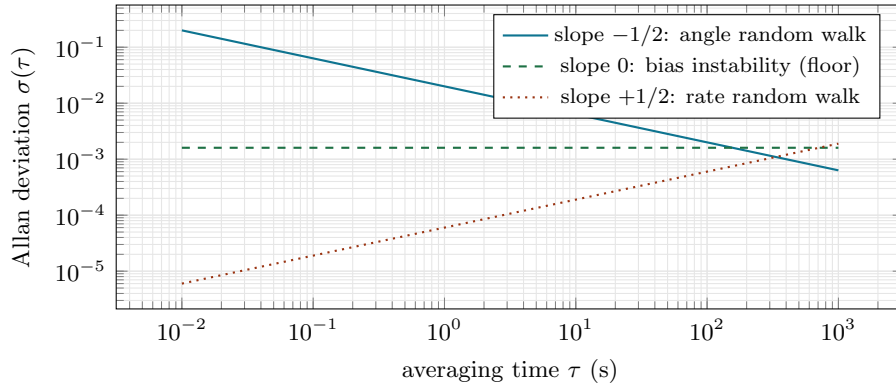
$$\mathbf{a}_c = -2\boldsymbol{\omega} \times \mathbf{v}.$$

This deflects the mass along the orthogonal *sense* axis by an amount proportional to ω , read capacitively. Because $\mathbf{a}_c \perp \mathbf{v}$ and $\perp \boldsymbol{\omega}$, a mass vibrating along x under rotation about z is pushed along y — that sensed motion *is* the angular-rate signal.

4.2 Bias, random walk, bias instability, Allan variance

The gyro's defining problem is **bias drift**. Integrating angular rate to get angle, a constant bias b_g yields heading error $\Delta\psi = b_g t$ (linear), while white rate noise yields **angle random walk**

(ARW) with $\sigma_\psi \propto \sqrt{t}$. The standard tool to separate these components is the **Allan variance** (El-Sheimy, Hou & Niu, 2008): a log-log plot of $\sigma(\tau)$ vs. averaging time τ whose *slopes* identify each noise type.



On the log-log Allan-deviation curve: a slope of $-1/2$ is **angle random walk** (white noise; read its coefficient N at $\tau = 1$); the flat minimum (slope 0) is **bias instability** (the irreducible floor of the gyro); a slope of $+1/2$ is **rate random walk**. These coefficients feed directly into the process-noise matrix $\mathbf{Q}$ of the Kalman filter (Phase 8).

4.3 Temperature, calibration, and gyro grades

Bias varies strongly with temperature (compensable with a $b(T)$ model). Grades span consumer MEMS (~ 10 – $100^\circ/\text{hr}$ bias), tactical (~ 1 – $10^\circ/\text{hr}$), and navigation-grade (fibre-optic/ring-laser, $< 0.01^\circ/\text{hr}$). Trinetra-AI targets low-cost MEMS, so *software* drift compensation is where the value lies.

The gyroscope is the backbone of short-term orientation in Trinetra-AI: it provides high-rate, low-latency rotation that the magnetometer (absolute heading) and accelerometer (gravity) correct over the long term. Its bias is *the* headline target of the drift-prediction module, because heading error leaks into position through the rotation matrix — a small gyro bias becomes a large position error over a GPS outage.

```
import numpy as np
def integrate_gyro(omega, dt, psi0=0.0): # 1-axis heading by dead reckoning
    return psi0 + np.cumsum(omega) * dt # bias here -> linear drift
def allan_deviation(x, fs, taus):
    """Overlapping Allan deviation of a rate signal x sampled at fs."""
    out = []
    for tau in taus:
        m = int(tau*fs); L = len(x)//m
        y = x[:L*m].reshape(L, m).mean(axis=1) # cluster averages
        out.append(np.sqrt(0.5*np.mean(np.diff(y)**2)))
    return np.array(out)
```

5 Magnetometer

5.1 Earth’s field, heading, declination, inclination

A **magnetometer** measures the local magnetic field vector $\mathbf{m} = [m_x, m_y, m_z]$ (μT). Near Earth’s surface this is dominated by the geomagnetic field ($\sim 25\text{--}65\ \mu\text{T}$). **Magnetic heading** is the horizontal direction of the field; **declination** δ is the angle between magnetic north and true (geographic) north; **inclination** (dip) is the angle the field makes with the horizontal.

Raw heading from the horizontal components is $\psi = \text{atan2}(-m_y, m_x)$, but only when the sensor is level. Using roll ϕ and pitch θ from the accelerometer, rotate the field into the horizontal plane:

$$\begin{aligned} m_x^h &= m_x \cos \theta + m_z \sin \theta, \\ m_y^h &= m_x \sin \phi \sin \theta + m_y \cos \phi - m_z \sin \phi \cos \theta, \end{aligned} \quad \psi_{\text{mag}} = \text{atan2}(-m_y^h, m_x^h).$$

True heading corrects for declination: $\psi_{\text{true}} = \psi_{\text{mag}} + \delta$, with δ from the World Magnetic Model (WMM) at the current location.

5.2 Hard-iron and soft-iron distortion

Ferromagnetic material on the platform distorts the reading. **Hard-iron** effects add a constant offset $\mathbf{b}$ (permanent magnets, DC currents); **soft-iron** effects scale and skew the field by a matrix $\mathbf{A}$ (induced magnetisation). The measured field is

$$\mathbf{m}_{\text{meas}} = \mathbf{A} \mathbf{m}_{\text{true}} + \mathbf{b} \implies \mathbf{m}_{\text{cal}} = \mathbf{A}^{-1}(\mathbf{m}_{\text{meas}} - \mathbf{b}).$$

Geometrically, rotating the sensor through all orientations should trace a *sphere* of radius $\|\mathbf{m}_{\text{true}}\|$; distortion turns it into an *ellipsoid*. Calibration fits that ellipsoid and maps it back to a sphere — recovering $\mathbf{A}$ and $\mathbf{b}$ (see §10).

5.3 Magnetic mapping, localization, and quantum magnetometers

Indoor magnetic fields are richly *distorted* by building steel — usually a nuisance, but here a gift: the distortion is spatially stable, forming a **magnetic map** to localise against (magnetic SLAM / map-matching). Outdoors, Earth’s *crustal anomalies* form a global fingerprint (the World Digital Magnetic Anomaly Map, WDMAM).

Why the magnetometer is the most important sensor for GPS-denied navigation. Unlike the gyro and accelerometer, whose integrated errors grow without bound, the magnetometer gives an *absolute*, drift-free position/heading cue by matching measured field to a pre-surveyed map. This is exactly the principle behind AQNav (SandboxAQ) and Ironstone Opal (Q-CTRL): a highly sensitive **quantum magnetometer** (NV-diamond or rubidium optical-vapour) reads nanotesla crustal anomalies as “landmarks,” and AI denoising removes the platform’s own magnetic signature — Boeing flew this GPS-free for

hours in 2024. In Trinetra-AI the magnetometer is the drift *anchor*; the future quantum channel simply makes that anchor far sharper without changing the pipeline.

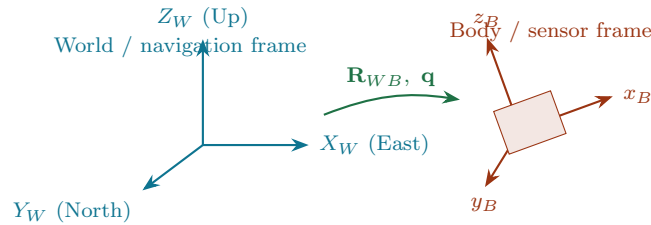
```
import numpy as np
def calibrate_mag(m, A_inv, b): # apply ellipsoid calibration
    return (A_inv @ (m - b).T).T
def tilt_compensated_heading(m, roll, pitch, decl=0.0):
    mx, my, mz = m
    mxh = mx*np.cos(pitch) + mz*np.sin(pitch)
    myh = (mx*np.sin(roll)*np.sin(pitch) + my*np.cos(roll)
           - mz*np.sin(roll)*np.cos(pitch))
    return np.arctan2(-myh, mxh) + decl # true heading (rad)
```

6 IMU

6.1 Definition and DOF

An **Inertial Measurement Unit (IMU)** packages inertial sensors in one device. **6-DOF** = 3-axis accelerometer + 3-axis gyroscope; **9-DOF** adds a 3-axis magnetometer (a *MARG* array); **10-DOF** adds a barometer. It outputs the raw streams that orientation and position estimation are built on.

6.2 Coordinate frames and rotation representations



Orientation maps the **body frame** to the **world (navigation) frame**. Three equivalent representations: **Euler angles** (yaw ψ , pitch θ , roll ϕ) — intuitive but suffer *gimbal lock*; the **rotation matrix** $\mathbf{R} \in SO(3)$ — 9 numbers, orthonormal; and the **quaternion** $\mathbf{q} = [q_w, q_x, q_y, q_z]$ — 4 numbers, singularity-free, the practical choice.

The ZYX Euler rotation matrix is $\mathbf{R} = \mathbf{R}_z(\psi)\mathbf{R}_y(\theta)\mathbf{R}_x(\phi)$. Its differential form — how orientation evolves under measured angular rate $\boldsymbol{\omega}$ — is cleanest as a quaternion ODE:

$$\dot{\mathbf{q}} = \frac{1}{2} \mathbf{q} \otimes \begin{bmatrix} 0 \\ \boldsymbol{\omega} \end{bmatrix},$$

where $\otimes$ is quaternion multiplication. Integrating this propagates attitude from the gyro; the accelerometer and magnetometer then correct the slow drift. Quaternions avoid the $\theta = \pm 90^\circ$ gimbal-lock singularity of Euler angles.

6.3 Dead reckoning and integration error

Strapdown mechanization integrates rate to attitude, rotates and integrates specific force to velocity, and integrates again to position:

$$\omega \xrightarrow{\int} \text{attitude} \rightarrow \mathbf{R} \mathbf{a} - \mathbf{g} \xrightarrow{\int} \mathbf{v} \xrightarrow{\int} \mathbf{p}.$$

Each integration accumulates its input's error: a gyro bias $\rightarrow$ heading error $\propto t \rightarrow$ mis-rotated gravity $\rightarrow$ velocity error $\propto t^2 \rightarrow$ position error $\propto t^3$ in the worst coupling; an accel bias alone gives $\propto t^2$. This unbounded growth is precisely why an IMU *must* be aided (GPS, magnetometer, baro) — the *raison d'être* of sensor fusion.

IMU data flow in Trinetra-AI: raw 9/10-DOF streams $\rightarrow$ time-sync $\rightarrow$ calibration (bias/scale/misalignment; mag hard/soft-iron) $\rightarrow$ de-noising (Phase 5) $\rightarrow$ windowing (Phase 4) $\rightarrow$ the AI modules (learned displacement, drift prediction) $\rightarrow$ the EKF/UKF fusion core, where the accelerometer+magnetometer bound the gyro's drift and the barometer bounds vertical error. The IMU is the high-rate spine; every other sensor exists to keep its integration honest.

```
import numpy as np
def quat_mul(q, r):
    w0,x0,y0,z0 = q; w1,x1,y1,z1 = r
    return np.array([w0*w1-x0*x1-y0*y1-z0*z1, w0*x1+x0*w1+y0*z1-z0*y1,
                    w0*y1-x0*z1+y0*w1+z0*x1, w0*z1+x0*y1-y0*x1+z0*w1])
def propagate_attitude(q, omega, dt): # integrate gyro into quaternion
    dq = 0.5 * quat_mul(q, np.concatenate([[0.0], omega]))
    q = q + dq*dt
    return q/np.linalg.norm(q) # renormalise
```

7 GPS

7.1 Architecture and positioning principle

The **Global Positioning System** (a GNSS) comprises a satellite constellation (each broadcasting its position and a precise time), a receiver, and ground control. The receiver measures the travel time of signals from ≥ 4 satellites and solves for its own 3-D position and clock offset by **trilateration** (ranging from known points) — not triangulation (angles).

The receiver cannot measure true range because its clock is imperfect; it measures a **pseudorange**

$$\rho_i = \|\mathbf{p} - \mathbf{s}_i\| + c \delta t_u + \epsilon_i,$$

where $\mathbf{p}$ is the unknown receiver position, $\mathbf{s}_i$ the i -th satellite position, δt_u the receiver clock bias, and c the speed of light. Four unknowns $(x, y, z, \delta t_u)$ require ≥ 4 satellites. The

nonlinear system is solved by linearising about a guess $\mathbf{p}_0$ and iterating least squares:

$$\Delta \rho = \mathbf{H} \begin{bmatrix} \Delta \mathbf{p} \\ c \Delta t_u \end{bmatrix}, \quad \mathbf{H}_i = \begin{bmatrix} -\hat{\mathbf{u}}_i^\top & 1 \end{bmatrix},$$

with $\hat{\mathbf{u}}_i$ the unit vector to satellite i . The geometry of the satellites sets the error amplification (dilution of precision, DOP).

7.2 Coordinates: ECEF, ENU, geodetic

Geodetic: latitude, longitude, altitude. **ECEF** (Earth-Centred, Earth-Fixed): Cartesian $[x, y, z]$ rotating with Earth. **ENU** (East-North-Up): a local tangent-plane frame at a reference point — the natural frame for navigation. NMEA is the ASCII message format receivers emit (e.g. \$GPGGA fixes).

Converting a geodetic fix to the local ENU frame (via ECEF) is what lets GPS be fused with the IMU’s local motion:

$$\mathbf{p}_{\text{ENU}} = \mathbf{R}_{\text{ENU}}^{\text{ECEF}} (\mathbf{p}_{\text{ECEF}} - \mathbf{p}_{\text{ref}}),$$

where $\mathbf{R}$ depends on the reference latitude/longitude.

7.3 Error sources — why GPS cannot be trusted alone

Error	Cause	Effect
Multipath	signal reflects off buildings/terrain	metre-level position jumps (non-Gaussian)
Atmospheric delay	ionospheric/tropospheric refraction	ranging bias
Clock errors	satellite & receiver clock offsets	absorbed as δt_u / residual
Ephemeris error	imperfect satellite position	ranging bias
Signal loss	indoors, tunnels, underwater	no fix at all
Urban canyon	blocked/reflected signals	poor geometry (high DOP), multipath
Jamming / spoofing	intentional interference	denial or deception

GPS is *absolute but fragile*: low-rate (1–10 Hz), unavailable indoors/underwater/in canyons, and jam/spoofable — which is the entire motivation for Trinetra-AI. In the pipeline GPS is the “truth when present, absent otherwise” channel: when available it resets inertial drift; when denied, the system falls back to IMU + magnetic-map + barometer fusion. Its multipath jumps are *non-Gaussian*, so they must be gated (Mahalanobis / innovation test) before entering the Kalman filter, or they corrupt the estimate.

```
import numpy as np
def geodetic_to_ecef(lat, lon, h, a=6378137.0, e2=6.69437999e-3):
    s = np.sin(lat); N = a/np.sqrt(1 - e2*s*s)
    x = (N+h)*np.cos(lat)*np.cos(lon); y = (N+h)*np.cos(lat)*np.sin(lon)
    z = (N*(1-e2)+h)*s
    return np.array([x,y,z])
def gps_gate(innov, S, thresh=9.0): # chi-square outlier rejection
    d2 = innov @ np.linalg.inv(S) @ innov # squared Mahalanobis distance
    return d2 < thresh # accept fix only if within gate
```

8 Barometer

8.1 Pressure, altitude, and the barometric formula

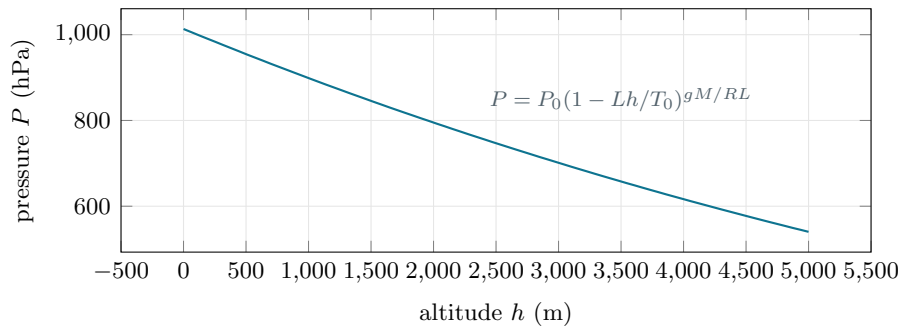
A **barometer** measures atmospheric pressure (hPa/Pa), typically via a MEMS piezoresistive or capacitive membrane that deflects under pressure. Because pressure falls predictably with height, it yields an **altitude** estimate — decoupled from horizontal motion.

Hydrostatic balance $dP = -\rho g dh$ with the ideal-gas law $\rho = PM/RT$ gives, for an isothermal layer,

$$\frac{dP}{P} = -\frac{gM}{RT} dh \Rightarrow P = P_0 \exp\left(-\frac{gM}{RT}h\right) \Rightarrow h = -\frac{RT}{gM} \ln \frac{P}{P_0}.$$

Using the standard-atmosphere lapse rate L (temperature $T = T_0 - Lh$) gives the form used in practice,

$$h = \frac{T_0}{L} \left[1 - \left(\frac{P}{P_0} \right)^{RL/gM} \right].$$



8.2 Drift, weather, calibration, noise

The barometer's weakness is **absolute** accuracy: weather changes P_0 over hours, so un-referenced barometric altitude drifts slowly. But its *relative* (short-term) altitude is excellent and low-noise. Calibration re-references P_0 to a known altitude (e.g. at takeoff, or from GPS when available).

How the barometer improves fusion: the vertical (z) axis of inertial navigation is the weakest — accelerometer double-integration drifts fastest there and GPS altitude is noisy. The barometer supplies an independent, low-noise vertical constraint that bounds this drift, dramatically stabilising altitude in the Kalman filter. In Trinetra-AI it is the cheap, always-available height anchor, especially valuable for UAVs and indoor/GPS-denied flight.

```
import numpy as np
def pressure_to_altitude(P, P0=1013.25, T0=288.15, L=0.0065,
                        g=9.80665, M=0.0289644, R=8.31447):
    return (T0/L)*(1 - (P/P0)**(R*L/(g*M))) # metres above the P0 reference
```

9 Sensor Errors

A unified taxonomy of the errors that corrupt every sensor, with how Trinetra-AI detects, corrects, and mitigates each — including where AI adds value over classical methods.

Error	Model	Detection	Correction / mitigation
Bias	constant offset b	static-mean estimate	subtract; estimate in KF state
Noise	σ^2 white	PSD / variance	filter, average, set $\mathbf{R}$
Drift	$b(t)$ slow trend	innovation trend, CUSUM	KF bias state; ML prediction
Random walk	$\propto \sqrt{t}$ variance	Allan variance	aiding sensors; $\mathbf{Q}$ tuning
Scale-factor	$(1+k)x$	multi-point calibration	apply gain correction
Misalignment	skew matrix $\mathbf{M}$	lab calibration	multiply by $\mathbf{M}^{-1}$
Temperature drift	$b(T), S(T)$	correlate with temp channel	$b(T)$ model / regression
Quantisation	step Δ , var $\Delta^2/12$	known from ADC bits	higher-bit ADC; dithering
Calibration error	residual $\mathbf{A}, \mathbf{b}$ error	residual analysis	re-calibrate; online estimation
Environmental / magnetic	external fields, EMI	spectral anomaly	shielding; hard/soft-iron cal
Mechanical vibration	band-limited noise	spectrogram peaks	notch/low-pass; isolation

AI-based correction is where Trinetra-AI goes beyond textbooks: learned models (LSTM/TCN) predict bias/drift ahead of time and pre-correct it (Brossard et al. 2020 denoise gyros with a dilated CNN); autoencoders flag anomalies by reconstruction error; and uncertainty heads output a confidence that becomes the filter’s adaptive $\mathbf{R}$. Classical methods handle the linear, stationary part cheaply; the network handles the structured, non-linear, temperature- and motion-dependent residual that classical models miss.

10 Sensor Calibration

10.1 Why, and the three regimes

Calibration determines a sensor’s error parameters (bias, scale, misalignment, hard/soft-iron) so raw readings can be corrected. **Factory** calibration is done once by the manufacturer; **offline** calibration is done by the user with controlled motions; **online** calibration estimates parameters continuously during operation (e.g. as extra states in the Kalman filter).

10.2 The general IMU calibration model

Each inertial triad is modelled as

$$\mathbf{z}_{\text{meas}} = \mathbf{T} \mathbf{K} (\mathbf{z}_{\text{true}}) + \mathbf{b} + \boldsymbol{\varepsilon},$$

where $\mathbf{K} = \text{diag}(k_x, k_y, k_z)$ are scale factors, $\mathbf{T}$ is a misalignment (non-orthogonality) matrix, and $\mathbf{b}$ is bias. Calibration estimates $\mathbf{T}, \mathbf{K}, \mathbf{b}$; correction applies $\mathbf{z}_{\text{cal}} = \mathbf{K}^{-1} \mathbf{T}^{-1} (\mathbf{z}_{\text{meas}} - \mathbf{b})$. The classic **six-position (tumble) test** places each accelerometer axis up/down against gravity to solve for bias and scale.

10.3 Magnetometer ellipsoid calibration

Uncalibrated magnetometer samples taken while rotating the sensor through all orientations satisfy the ellipsoid

$$(\mathbf{m} - \mathbf{b})^T \mathbf{M} (\mathbf{m} - \mathbf{b}) = 1,$$

a general quadric fitted by least squares from the samples. Diagonalising $\mathbf{M} = \mathbf{A}^{-T} \mathbf{A}^{-1}$ recovers the soft-iron matrix $\mathbf{A}$ and hard-iron offset $\mathbf{b}$, and the correction $\mathbf{m}_{\text{cal}} = \mathbf{A}^{-1} (\mathbf{m} - \mathbf{b})$ maps the ellipsoid back to a unit sphere. **This is the single most important calibration for GPS-denied heading**, because an uncalibrated magnetometer gives a heading that is wrong in a direction-dependent way.

```
import numpy as np
def fit_hard_soft_iron(M): # M: (N,3) samples over all orientations
    # least-squares ellipsoid fit -> returns offset b and matrix A_inv
    x,y,z = M[:,0],M[:,1],M[:,2]
    D = np.column_stack([x*x,y*y,z*z,2*x*y,2*x*z,2*y*z,2*x,2*y,2*z,np.ones_like(x)])
    _,_,V = np.linalg.svd(D); p = V[-1] # null-space solution
    # (assemble quadric, solve center b and shape A_inv; abbreviated)
    return p # full unpacking in the notebook
def gyro_bias_static(omega_static): # simplest online-ready bias estimate
    return omega_static.mean(axis=0)
```

11 Sensor Drift

Drift — the slow, unbounded growth of error — is the central obstacle of GPS-denied navigation and the reason Trinetra-AI exists. Each sensor drifts differently.

Sensor	Drift character	Dominant correction
Gyroscope	bias walk $\rightarrow$ heading err. $\propto t$	magnetometer + accel aiding; ML bias predict
Accelerometer	bias $\rightarrow$ position err. $\propto t^2$	gravity reference; KF bias state
Magnetometer	environmental field change	re-calibration; map-matching
GPS	multipath “jumps” (not slow drift)	outlier gating; better geometry
Barometer	weather-driven P_0 drift	re-reference to GPS/known altitude

Correction family	Method & role in Trinetra-AI
Traditional Kalman-based	high-pass filtering, temperature regression $b(T)$; cheap, linear augment state with bias; innovation-driven online estimation (Phase 8)
Adaptive	adaptive $\mathbf{Q}/\mathbf{R}$, LMS/RLS tracking of changing conditions
Machine learning	regression/one-class models on features; fault & health flags
Deep learning	LSTM/TCN forecast bias ahead of time and pre-correct (the drift module)

Trinetra-AI’s stance: the Kalman filter estimates bias *reactively* through innovations; the learned drift module predicts it *proactively* from temporal + temperature features and subtracts it before integration. Together they attack the fastest-growing error terms (t for the gyro, t^2 for the accelerometer) that dominate a GPS outage — and the magnetometer/map provides the absolute anchor that resets whatever drift remains.

12 Sensor Datasets

Real, verified datasets; confirm licence/download on the official page before use. Difficulty is relative to a solo research project.

Dataset	Sensors	Ground truth	Diff.	Best for
OxIOD	IMU (accel/gyro/-mag)	Vicon/VI	Easy–Med	inertial odometry (Chen et al.)
RoNIN	IMU (phone)	3D traj.	Med	neural inertial nav (Herath et al., ICRA 2020)
TLIO	IMU (headset)	VIO	Med	learned displacement + EKF (Liu et al., 2020)
EuRoC MAV	IMU + stereo	Vicon/Leica	Med	drone VIO (Burri et al., 2016)
KITTI	IMU+GPS/RTK+LiDAR	RTK+GPS/INS	Med	autonomous driving (Geiger et al.)
TUM VI	IMU + stereo	partial MoCap	Med	precise IMU-cam sync (Schubert et al., 2018)
comma2k19	IMU+GNSS+CAN	GNSS+fusion	Med	GPS/INS driving fusion
KAIST Urban	IMU+GPS+LiDAR	fused	Hard	urban-canyon multi-path
MagPIE	IMU + magnetometer	cm-level	Med–Hard	magnetic positioning (Hanley et al., 2017)
DAF-MIT AIA MagNav	aeromagnetic + INS	flight ref.	Hard	magnetic-anomaly navigation (Gnadt et al., 2023; Zenodo)
UCI HAR	phone IMU (no mag)	activity labels	Easy	pipeline/windowing practice

Synthetic option. Simulate a magnetometer along a ground-truth path from the World Magnetic Model (WMM), or generate full IMU+mag+baro streams in Gazebo/AirSim for controlled GPS-denial ablations with perfect ground truth. The **DAF-MIT AIA** set is the purpose-built resource for the magnetic-anomaly (quantum-navigation) side of the project.

13 Research Papers

Verified, correctly attributed. Priority: ★★★ read first, ★★ core, ★ as needed.

13.1 Foundational — inertial sensors & navigation

Work	Authors, year	Contribution	Pri.
Strapdown Inertial Navigation Technology	Titterton & Weston, IET 2004	the standard INS textbook	★★★
Analysis & Modeling of Inertial Sensors Using Allan Variance	El-Sheimy, Hou & Niu, IEEE T-IM 2008	noise identification → Q, R	★★★
MEMS Inertial Sensors: A Tutorial Overview	Shaeffer, IEEE Comm. Mag. 2013	MEMS accel/gyro physics	★★
Principles of GNSS, Inertial & Multi-sensor Integrated Navigation	Groves, 2013	GNSS/INS integration reference	★★
Estimation of IMU/MARG Orientation (Madgwick filter)	Madgwick, Harrison & Vaidyanathan, ICORR 2011	lightweight orientation filter	★★
Visual-Inertial Navigation: A Concise Review	Huang, ICRA 2019	modern VIN survey	★

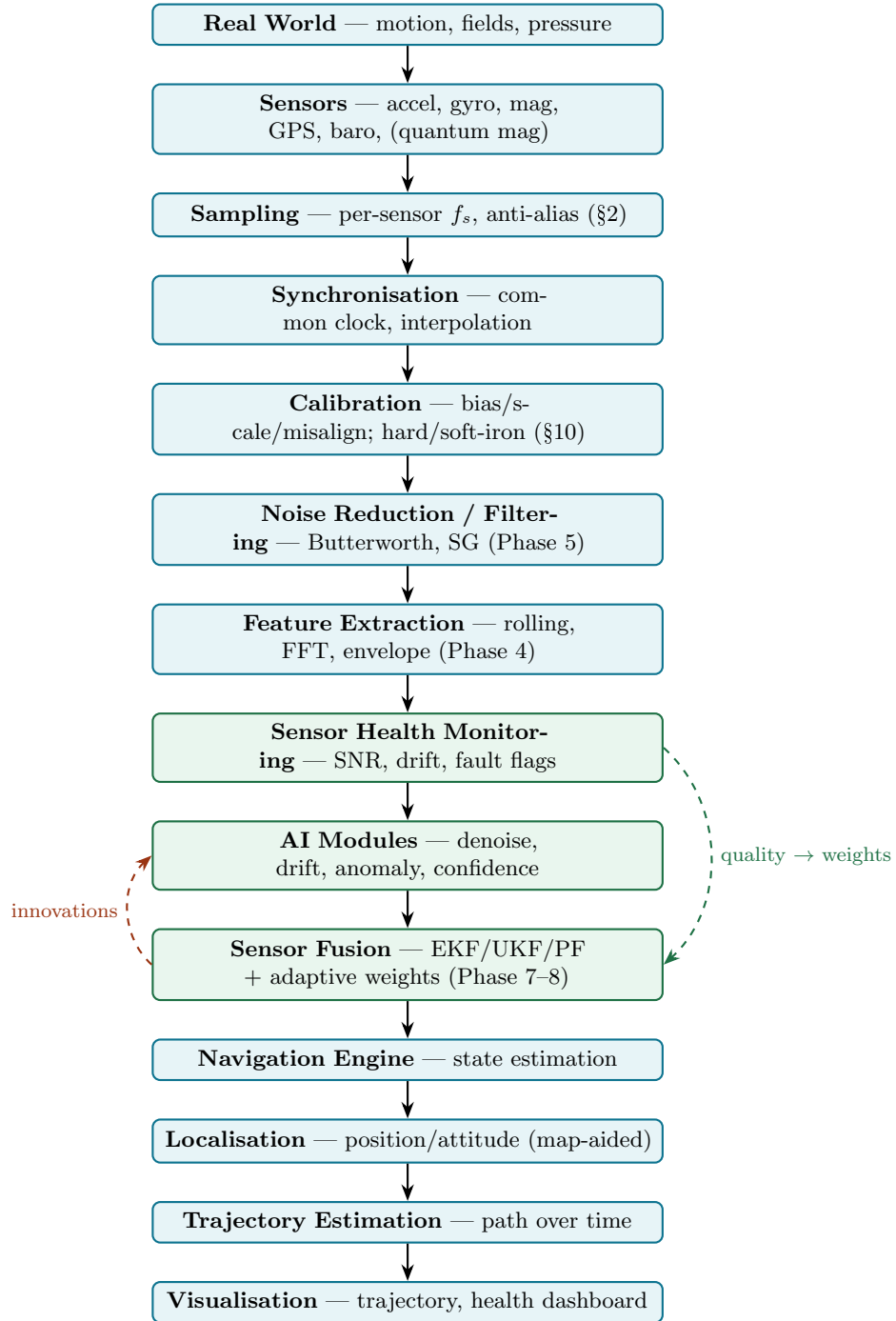
13.2 Learned inertial & magnetic navigation (most relevant)

Paper	Authors, year	Helps Trinetra-AI	Pri.
IONet	Chen et al., AAAI 2018	first DNN inertial odometry	★★★
RoNIN	Herath et al., ICRA 2020	neural inertial nav + dataset/metrics	★★★
TLIO	Liu et al., RA-L 2020	learned displacement + uncertainty in EKF	★★★
Denoising IMU Gyroscopes w/ DL	Brossard, Bonnabel & Barrau, RA-L 2020	dilated-CNN gyro denoising	★★
DAF-MIT AIA Open Flight Data for MagNav	Gnadt et al., 2023	open magnetic-navigation benchmark	★★
Quantum-assured magnetic navigation (Ironstone Opal)	Muradoglu et al., 2025 (arXiv:2504.08167)	quantum-magnetometer nav beating strategic INS	★★

Reading path: Titterton & Weston (physics) → El-Sheimy (noise modelling) → IONet/RoNIN/TLIO (learned inertial) → Gnadt (MagNav data) → Muradoglu (quantum MagNav) traces the exact arc from classical sensors to the quantum-navigation frontier Trinetra-AI targets. **Citation hygiene:** confirm every DOI/venue on the official page before it enters a report — the industry milestones (Boeing 2024 quantum flight; SandboxAQ AQNav; Q-CTRL Ironstone Opal) are best cited from primary press releases / papers, not secondary summaries.

14 Sensors inside Trinetra-AI

The full sensor pipeline, from the physical world to the navigation output. The dashed loops are the intelligence: health/quality tunes fusion forward; innovations inform health backward.



Block detail. *Sampling/sync* put multi-rate sensors on one timeline; *calibration* removes systematic error; *filtering* (Phase 5) removes noise; *feature extraction* (Phase 4) feeds *health monitoring* and the *AI modules*, which supply corrected measurements, adaptive $\mathbf{Q}$, $\mathbf{R}$, and per-sensor weights to *fusion* (Phases 7–8); the fused state drives *localisation* and *trajectory*, closing the loop through innovations.

15 Practical Implementation

Notebook	Contents	Expected output
01_read_sensors.ipynb	load CSV/ROS bags; parse NMEA	unified sensor DataFrame
02_sync_resample.ipynb	timestamp align, interpolate	uniform multi-rate arrays
03_calibrate.ipynb	accel 6-pos, gyro bias, mag ellipsoid	calibration params T, K, b, A
04_noise_allan.ipynb	PSD, Allan variance	noise coefficients for Q, R
05_orientation.ipynb	roll/pitch/heading, quaternion	attitude time series
06_deadreckoning.ipynb	strapdown integration	drifting trajectory (baseline)
07_health_features.ipynb	SNR, rolling stats, fault flags	per-sensor health signal
08_store_prepare_ai.ipynb	Parquet store; windowed tensors	(N, L, C) arrays for models

Reading & preprocessing. Use pandas/NumPy to load and resample; SciPy for filtering; parse GPS NMEA with `pynmea2`. **Store** raw + processed as Parquet (compact, typed) and keep a raw copy for audit. **Prepare for AI** by calibrating, de-noising, normalising per channel, and windowing (Phase 4).

Visualisation. Time-series overlays (raw vs calibrated), 3-D orientation, Allan-deviation log-log plots, magnetometer sphere/ellipsoid before-after, trajectory vs ground truth.

Evaluation metrics. Bias/scale residual after calibration, noise density (Allan), heading RMSE (deg), dead-reckoning drift rate (m per 100 m), altitude RMSE, and — end-to-end — downstream ATE/RTE once fused.

Common mistakes. Skipping magnetometer hard/soft-iron calibration (heading wrong in a direction-dependent way); ignoring axis misalignment; estimating noise on *moving* data; forgetting declination; not gating GPS multipath; mismatched units (deg vs rad, g vs m/s^2). **Best practices.** Characterise noise on *static* data; calibrate every sensor before fusion; version calibration params; unit-test coordinate transforms; always keep a raw copy and a well-tuned classical dead-reckoning baseline.

16 Interview & Research Preparation

16.1 50 interview questions

1. Define a sensor as a transfer function.
2. Accuracy vs precision (sensor example).
3. Why does averaging cut noise by $\sqrt{N}$ but not bias?
4. Bias-variance of a reading.
5. Sensitivity vs resolution.
6. Scale-factor error.
7. Dynamic range in dB.
8. Bandwidth vs response time.
9. Nyquist for a sensor.
10. Latency in fusion.
11. Hysteresis vs dead zone.
12. Quantisation-noise variance.
13. Uncertainty propagation through integration.

14. What does an accelerometer actually measure?
15. Reading at rest vs free-fall.
16. Spring-mass transducer equation.
17. Sensitivity vs bandwidth trade-off.
18. Capacitive vs piezoelectric accel.
19. Roll/pitch from accel; why not yaw?
20. Accel bias $\rightarrow$ position error growth.
21. What does a gyro measure?
22. Coriolis principle in a MEMS gyro.
23. Gyro bias drift $\rightarrow$ heading error.
24. Angle random walk.
25. Allan-variance slopes.
26. Bias instability.
27. Gyro grades (consumer/tactical/nav).
28. Earth field magnitude & components.
29. Declination vs inclination.
30. Tilt-compensated heading.
31. Hard-iron vs soft-iron.
32. Ellipsoid calibration idea.
33. Why magnetometer for GPS-denied nav?
34. Magnetic map-matching.
35. 6/9/10-DOF IMU.
36. Body vs world frame.
37. Euler angles & gimbal lock.
38. Rotation matrix properties.
39. Quaternion advantages.
40. Quaternion kinematics.
41. Dead-reckoning error growth.
42. GPS trilateration vs triangulation.
43. What is a pseudorange?
44. Why ≥ 4 satellites?
45. ECEF vs ENU.
46. GPS multipath & why it breaks a KF.
47. Why GPS fails indoors.
48. Barometric altitude formula.
49. Barometer drift cause.
50. How the barometer stabilises vertical fusion.

16.2 30 viva questions

1. Derive the spring–mass accelerometer response.
2. Derive Coriolis force in a gyro.
3. Explain each Allan-variance slope and its coefficient.
4. Derive tilt-compensated heading.
5. Set up magnetometer ellipsoid calibration.
6. Write the IMU bias–scale–misalignment model.
7. Derive quaternion attitude kinematics.
8. Show why gimbal lock happens.
9. Trace dead-reckoning error growth (t, t^2, t^3) .
10. Set up GPS pseudorange least squares.
11. Explain DOP.
12. Convert geodetic $\rightarrow$ ECEF $\rightarrow$ ENU.
13. Derive the barometric formula.
14. How do you get **Q**, **R** from Allan variance?
15. Why calibrate the magnetometer before fusion?
16. Static vs dynamic noise characterisation.
17. Detect and gate GPS multipath.
18. Complementary filter: which band from which sensor?
19. Where do AI modules beat classical correction?
20. Design a six-position accel calibration.
21. Temperature-compensation model.
22. Explain the full sensor pipeline.
23. Which sensor anchors drift and why?
24. Role of the barometer in the vertical channel.
25. How does a quantum magnetometer change the design?
26. Failure modes of learned correction.
27. Why is misalignment a matrix, not a scalar?
28. How does latency bias fusion?
29. Units pitfalls (deg/rad, $g/\text{m s}^{-2}$).
30. Validate a calibration.

16.3 25 research questions

1. Can learned bias prediction beat KF innovation estimation over long outages?
2. Optimal window length for drift prediction vs latency.
3. Quantum-magnetometer drift-reduction ceiling (simulated).
4. Robustness of magnetic map-matching to environmental change.
5. Temperature-conditioned vs agnostic drift models.
6. Learned vs classical magnetometer calibration.

7. Online vs offline calibration accuracy trade-off.
8. Uncertainty-aware fusion (TLIO-style) gains.
9. Cross-platform generalisation of denoising models (cf. MagNav).
10. Minimal sensor set for a target accuracy.
11. Fusion-loop stability when ML is wrong.
12. Detecting spoofed GPS with a learned model.
13. Effect of sync error on fusion accuracy.
14. Barometer+IMU vertical fusion limits.
15. Allan-variance-informed vs empirical $\mathbf{Q}$.
16. Wheel-encoder + IMU odometry vs IMU-only.
17. Camera/LiDAR aiding for GPS-denied loop closure.
18. Atomic-clock aiding of GNSS/INS.
19. Magnetic-anomaly map resolution vs localisation accuracy.
20. Edge-deployable calibration on Jetson.
21. Sensor-health scoring calibration (reliability).
22. Data-efficient training for new sensors.
23. Domain gap: simulated vs real IMU.
24. Fault-injection robustness testing.
25. Quantum-noise (shot-noise) modelling in fusion.

16.4 20 coding questions

1. Read and plot a 9-DOF IMU CSV.
2. Resample IMU + GPS onto one timeline.
3. Estimate accel bias/noise from static data.
4. Compute roll/pitch from accel.
5. Integrate gyro to heading; show drift.
6. Compute overlapping Allan deviation.
7. Fit magnetometer hard/soft-iron and re-plot the sphere.
8. Tilt-compensated heading with declination.
9. Quaternion propagation from gyro.
10. Detect gimbal lock in Euler integration.
11. Strapdown dead-reckoning of a short trajectory.
12. Parse NMEA and extract lat/lon/alt.
13. Geodetic $\rightarrow$ ENU conversion.
14. Chi-square gate for GPS multipath.
15. Pressure $\rightarrow$ altitude with the barometric formula.
16. Butterworth-filter a noisy accel axis.
17. Spectrogram of vibration for fault detection.
18. Build windowed tensors for the AI models.
19. Store sensor data as Parquet and reload.
20. Compute dead-reckoning drift-rate metric vs ground truth.

17 Summary, Glossary & Roadmap

17.1 Summary

Sensors are the foundation of the whole system: each converts a physical quantity into a signal that lies in characteristic ways (bias, noise, drift). The accelerometer gives gravity/linear acceleration, the gyroscope high-rate rotation (but drifts), the magnetometer an absolute drift-free heading and a map to localise against, GPS an absolute-but-fragile fix, and the barometer a low-noise vertical anchor. Understanding each sensor's physics and error model is what lets you calibrate it, correct its drift, and — above all — hand the fusion filter the honest covariances it needs. The magnetometer (soon quantum) is the drift anchor that makes GPS-denied navigation viable.

17.2 Glossary

ARW — angle random walk ($\sqrt{t}$ heading noise). **Allan variance** — log-log tool separating sensor noise terms. **Bias / drift** — offset / its slow time-variation. **Coriolis effect** — force on a mass moving in a rotating frame; how MEMS gyros sense. **Dead reckoning** — position by integrating motion from a known start. **Declination / inclination** — angle to true north / dip of the field. **DOF** — degrees of freedom (6/9/10-DOF IMU). **ECEF / ENU** — Earth-fixed / local tangent frames. **Hard/soft-iron** — additive / ellipsoidal magnetometer distortion. **MARG** — Magnetic, Angular Rate, Gravity sensor array. **Pseudorange** — GPS range including receiver clock bias. **Quaternion** — 4-number singularity-free attitude representation. **Scale factor / misalignment** — gain error / axis non-orthogonality. **Specific force** — what an accelerometer measures (accel minus free-fall).

17.3 Roadmap: Phase 6 → Phases 7, 8, 9 & the Trinetra-AI architecture

Next phase	What this phase hands it
Phase 7 — Sensor Fusion	calibrated, time-synced, health-scored measurements + per-sensor error models to combine
Phase 8 — Kalman Filter	the noise covariances $\mathbf{Q}, \mathbf{R}$ (from Allan variance), bias states, and the measurement models $h(\cdot)$
Phase 9 — Navigation Systems	the dead-reckoning core, magnetic-map anchor, and vertical (baro) constraint that define GPS-denied operation
Trinetra-AI architecture	the full sensor front-end feeding the AI modules and fusion core — with the quantum magnetometer as a drop-in future channel

Enter Phase 7 knowing not just *what* each sensor outputs, but *how it lies* — because sensor fusion is, at its core, the disciplined combination of several imperfect, well-characterised liars into one estimate better than any of them alone. Everything you calibrated and modelled here becomes the $\mathbf{Q}, \mathbf{R}, h(\cdot)$, and bias states of the Kalman filter to come.

End of Phase 6 notes. Next: Phase 7 (Sensor Fusion) → Phase 8 (Kalman Filter) → Phase 9 (Navigation Systems) → the final Trinetra-AI architecture.